

WHY search

A tutorial for humans - plain-English walkthrough of what we built, and why

A friendly walkthrough of what we built, why we built it that way, and every piece of jargon translated into plain English.

If you're reading this and thinking "*I don't know what any of this stuff means*," that's the point. Every technical word is explained the first time it appears, with a real-world comparison. No prior coding or ML background is assumed.

The one-sentence version

WHY search is a small search engine that looks through 420 research papers and 412 open-source projects at the same time, and for every result, tells you exactly *why* it picked that result - either the specific words that matched or, using AI, the fact that the meaning matched.

That's it. The rest of this document explains what that means.

Why is this project interesting?

You already use search engines every day - Google, YouTube search, Amazon's search bar, Netflix's "browse" screen. They all do the same thing: take some words you typed and find things that are related to those words.

But here's a fun question: **how does Google actually decide that one page is more related to your query than another?** Most people have no idea. The results just appear.

This project builds a working search engine from scratch - small enough to understand every line, useful enough to actually find things. It uses two of the same core techniques that Google, Netflix, and Spotify use internally:

1. **TF-IDF** - a technique from the 1970s that ranks documents by which words overlap with your query. Still the foundation of every modern search engine.
2. **Semantic embeddings** - a modern AI technique that understands *meaning* instead of just words, so "chatbot" can find a project called "conversational agent" even though they share no letters.

And it does one thing on top that regular Google doesn't:

3. **It shows you why it picked each result.** Highlighted words, ranked by how much they contributed. This is the "WHY" in WHY search.

It's inspired by research from **Prof. Jie Zou at UESTC** on making information retrieval systems more transparent and conversational. Building this project was our way of learning that field by touching every piece of the pipeline.

What we built, step by step

We built the project in 7 small stages. Each one is a separate Python file, so you can open any one of them and see exactly what it does. Let's walk through them in the order we built them.

Some vocabulary you'll need in every step:

What's a script? A `.py` file with instructions the computer runs top to bottom. Like a recipe.

What's an API? A doorway a website leaves open for programs to talk to it directly, instead of you clicking around in a browser. arXiv has one, GitHub has one, Twitter has one. You send a URL, they send back the data.

What's JSON? A plain-text way of writing structured information so any programming language can read it. It looks like this: `{"title": "cats", "views": 42}`. Whenever you see a `.json` file, think of it as an organized notebook of records.

Step 1: Grab research papers from arXiv

- **File:** `fetch_arxiv.py`
- **Runs by:** `python fetch_arxiv.py`
- **What it produces:** `data/papers.json`

arXiv (pronounced "archive" - the X is a Greek letter chi) is a giant open library of research papers. It has millions of scientific papers you can read for free, and every one of them has a public API you can query.

Our script politely asks arXiv's API for papers about three topics:

- *information retrieval* - the science of search engines
- *recommender systems* - the science behind "you might also like..."
- *natural language processing* - teaching computers to understand human language

For each topic we ask for 140 papers. The API answers in a language called XML (think of it as a stricter cousin of JSON), and our script parses out the title, abstract, URL, and publication date of each paper.

What we ended up with: 420 unique papers. (We asked for 420 total across three topics; after removing duplicates that showed up in more than one topic, we still had 420 - clean run.)

A quirk worth noting: the very first run failed with an "SSL certificate" error. That's a security-check problem some Windows Python installs have - the computer didn't know how to verify that arXiv's website is really arXiv's website. We fixed it by installing `certifi`, a tiny package whose only job is to carry an up-to-date list of trusted certificates. Small annoyance, common on Windows, one-line fix.

Step 2: Grab open-source projects from GitHub

- **File:** `fetch_github.py`
- **Runs by:** `python fetch_github.py`
- **What it produces:** `data/repos.json`

GitHub is where nearly every open-source coding project lives. Each project (called a "repository," or "repo" for short) has a page with its code, a description, and a README - a text document the authors write to introduce the project. GitHub also has an API.

Our script searches GitHub for repos tagged with the same three topics as before:

`recommender-system`, `information-retrieval`, `nlp`. It grabs the top ~170 most-starred repos for each topic. Then for each repo, it fetches the README text.

What we ended up with: 412 repos. Fewer than the 500 we aimed for, because many top repos are tagged with more than one of our topics (HuggingFace's `transformers` project matched all three), so dedup collapsed them. That's genuine overlap, not a rate-limit hit - recorded honestly.

A trick used here: GitHub's API has strict limits on how many README fetches you can do per hour without logging in. Instead, we grabbed the raw README text from `raw.githubusercontent.com` - the same file, different URL, much more relaxed limits. This kind of small workaround is what makes the difference between a script that finishes in 5 minutes and one that dies after 60 requests.

Step 3: Merge both into one dataset

- **File:** `build_dataset.py`
- **Runs by:** `python build_dataset.py`
- **What it produces:** `data/combined.json`

Papers and repos live in two very different formats. This tiny script mashes them into one unified list where every entry looks the same:

```
{
  "text": "the searchable content",
  "source_type": "paper" or "repo",
  "title": "...",
  "url": "..."
}
```

For a paper, `text` is `title + abstract`. For a repo, `text` is `name + description + README`. That way the next step (the search index) can treat both the same way - it doesn't need to know

whether an entry is a paper or a repo when computing similarity.

What we ended up with: 832 entries (420 papers + 412 repos). This is our search index's raw material.

Step 4: Build the keyword search (TF-IDF)

- **File:** `search.py` (function `SearchIndex.search`)
- **What it produces:** rankings when you call `.search("query")`

Here's where the actual search brain lives. Let me explain the core idea before showing what we did.

The problem: You have 832 documents. Someone types "conversational search." How do you decide which document is the best match?

The naive approach - count how many times each word from the query appears in each document, pick the document with the highest count - is broken in two ways:

1. Long documents "win" just by being long. A giant README will contain almost every English word by accident.
2. Common words like *the*, *is*, and *and* would dominate the count without adding meaning.

TF-IDF fixes both. It's an acronym for **Term Frequency - Inverse Document Frequency**. In plain English:

- **Term Frequency (TF):** how often the word appears in *this document*. More = stronger signal. But we take the log so that a word appearing 100 times isn't judged 100× more important than appearing 10 times.
- **Inverse Document Frequency (IDF):** how *rare* the word is across the whole collection. A word that appears in 1 of 832 documents is a strong specific signal ("diffusion") - much more informative than a word that appears in 800 out of 832 ("model").

Multiply them together, and every word in every document gets a **weight** that says "how important is this word to this document, on this dataset?"

Now every document becomes a **vector**: a long list of numbers, one per word in our vocabulary. Our vocabulary has 25,207 unique words and 2-word phrases (called "bigrams"), so each document is a vector of 25,207 numbers, most of which are zero (because most words don't appear in most documents).

When you type a query, we do the exact same thing to your query - turn it into a 25,207-long vector - and then measure how "close" your query vector is to each document vector using **cosine similarity**.

What is cosine similarity? Imagine every document is an arrow pointing somewhere in a giant space. Two arrows pointing the same direction = the same topic. Two arrows at right angles = unrelated. Cosine similarity is a number from 0 (completely unrelated) to 1 (identical) that measures the angle between them. It's used everywhere in search and recommendation systems.

What we ended up with: a search that runs in milliseconds even with an old laptop, no GPU, no API, no internet after the initial data fetch. It's classical, but it's fast, robust, and predictable.

Step 5: Add the "why it matched" explanation

- **File:** [search.py](#) (method `_overlapping_terms`)

Here's the trick that's the whole point of "**WHY** search."

Cosine similarity between two vectors is technically just a big **dot product** - you multiply each pair of matching numbers and add them all up. That means each word's contribution to the final score is a single number you can pull out.

So we do exactly that: **element-wise multiply the query vector by the document vector**, and every nonzero result tells you "this word contributed *this much* to the match." We sort those, pull out the top 6, and that's what you see in the yellow chips on each result card.

That's it. No fuzzy explanation, no machine-learning interpretability layer. Just a math trick that falls out of how TF-IDF already works. Nothing is invented, nothing is guessed - it's the actual scoring mechanism, shown to the user.

This is a small deal in a technical sense, but a big deal in an interview sense - most search systems today are opaque black boxes, and being able to say "my system tells you *exactly* why every result was picked" is a real answer to a real problem.

Step 6: Add semantic search (understanding meaning)

- **New file:** [build_embeddings.py](#)
- **Extended file:** [search.py](#) (methods `semantic_search`, `hybrid_search`)

TF-IDF has one big weakness. If your query says "chatbot" and a document says "conversational agent," TF-IDF has no idea those are related. They share zero letters. It's a fundamentally letter-matching approach.

Semantic search fixes this - it matches on *meaning* instead of on letters.

How? Using an **AI model** (specifically, a small one called `all-MiniLM-L6-v2` from a group called sentence-transformers) that has read a giant chunk of the internet. In the process it learned which words tend to appear in similar contexts. When you feed it a sentence, it hands you back an **embedding** - a list of 384 numbers that represents the *meaning* of that sentence.

Two sentences with similar meaning get similar numbers, even if they share zero words.

The analogy: Imagine every possible sentence gets a GPS coordinate on a giant map of "meaning space." "Chatbot," "conversational agent," and "dialogue system" all end up in the same neighborhood. To search, you find the coordinate of your query, then find the documents whose coordinates are closest. That's it: **text -> numbers -> find nearest neighbors**.

The script `build_embeddings.py` runs once. It:

1. Downloads the model (~90 MB, cached forever after) from Hugging Face - a website that hosts free AI models the way GitHub hosts free code.
2. Feeds all 832 documents through the model.
3. Saves the resulting embeddings as a small numeric file (`data/embeddings.npy` , only 1.2 MB - because it's just numbers, not text).

Now when you type a query, we embed *the query* the same way (takes about 200 ms) and find the closest documents. That's semantic search.

Hybrid mode is what you see when the "Hybrid" toggle is on in the app. It runs *both* keyword and semantic search at once, side by side. That way you can compare - for the query "chatbot," the keyword column will find papers that literally say "chatbot," and the semantic column will find famous conversational-agent frameworks (DeepPavlov, RasaHQ, Onyx) that never contain the word "chatbot" but do the thing.

This side-by-side comparison is the most interesting part of the app to look at. It shows you, viscerally, the difference between the two techniques.

Step 7: Wrap it all in a Streamlit web app

- **File:** `app.py`
- **Runs by:** `python -m streamlit run app.py`
- **What it does:** the UI you saw in your browser

Everything above is happening in Python scripts. Great for us, useless for anyone else. To turn it into a real usable app, we used **Streamlit**.

The next section explains what Streamlit is, why we picked it, and what it can and can't do.

Why Streamlit? (and what it can do that Vercel can't)

What Streamlit is, in one sentence

Streamlit is a tool that turns a Python script into a web app, without you writing any HTML, CSS, or JavaScript.

Normally, building a web app is a two-language project: you write your data-processing logic in Python (or another server language), then you write a totally separate frontend in HTML+CSS+JavaScript to display it. Two languages, two teams' worth of knowledge, days of glue code.

Streamlit says: **just write Python**. You write a script top-to-bottom, using calls like `st.text_input()` , `st.button()` , `st.dataframe()` , and Streamlit turns those into real interactive widgets in a real web page.

Here's a working Streamlit app in 6 lines:

```
import streamlit as st

name = st.text_input("What's your name?")
if name:
    st.write(f"Hi, {name}! !")
```

Save that as `hello.py`, run `streamlit run hello.py`, and you have a live web page with a text input and dynamic response. That's genuinely all there is to it.

How Streamlit actually works (the mental model)

This is the one thing that makes Streamlit feel weird at first, and clicks the moment you understand it.

Every time anything changes on the page - you type a letter, click a button, move a slider - Streamlit re-runs your entire Python script from top to bottom.

Not just the affected part. The whole script. Every time.

That sounds insane, but it's the trick that makes Streamlit simple: you never have to reason about state, events, or updates. Your script just says "given the current state of the widgets, here's what the page should look like." Streamlit runs it, diffs the output, and updates only what changed on the screen.

To keep it fast, Streamlit has a `@st.cache_resource` decorator that lets you say "this expensive thing (loading a model, building an index) - remember it, don't rebuild it every rerun." That's how our app can load 90 MB of AI weights once and never again during a session.

Why we used it for this project

- **The whole search engine is Python.** TF-IDF (scikit-learn), embeddings (sentence-transformers), the dataset (JSON files) - all Python. Building a Python-native UI for a Python project is the shortest possible path.
- **No frontend rewrite.** We don't have to reinvent our search logic in JavaScript.
- **Sharing is free.** Streamlit has a hosting service - `share.streamlit.io` - that's free forever for public apps. You push your GitHub repo, they give you a live URL. 3 minutes end-to-end.
- **AI-friendly.** Streamlit was built specifically for data scientists and ML engineers showing off their work. The moment you say "I want to load a model and let people query it," Streamlit is the default answer.

What Streamlit can do that Vercel can't

Vercel is a hosting service you probably heard about because it's what everyone uses for React/Next.js sites - the polished, animated marketing pages. It's beautiful for that. But Vercel isn't designed to run Python programs that hold big things in memory.

Here's the specific gap:

	Streamlit Cloud	Vercel
Run a Python script with heavy dependencies (PyTorch, scikit-learn, AI models)	[yes] Yes, built for it	[partial] You'd have to put the Python somewhere else and call it from Vercel
Keep a large model in memory across requests	[yes] Yes (<code>@st.cache_resource</code> - model loads once, stays there)	[no] No - each request is a fresh short-lived function; the model would reload every time
Zero frontend code	[yes] Yes	[no] You write React + HTML + CSS
Free hosting	[yes] Yes for public apps	[yes] Yes for hobby projects
Load time on first visit	~5 seconds if the app was sleeping	Instant
Custom design freedom (fonts, animations, layouts)	Limited (Streamlit's look with some CSS tweaks)	Total
Best for	ML/data apps, dashboards, internal tools, portfolio demos of <i>models</i>	Marketing sites, landing pages, e-commerce, portfolio demos of <i>design</i>

In plain words: Streamlit is the right choice when your project's magic is what happens on the *server* (searching, ranking, running a model). Vercel is the right choice when your project's magic is what happens in the *browser* (animations, polished visual design).

For a keyword+semantic search engine with a 90 MB AI model that has to stay in memory, Streamlit is genuinely the better fit - not just the easier one.

What Streamlit isn't great at

Being honest, because you'll want to know:

- **Sub-second interactive animations.** Every interaction reruns the whole script, so anything requiring smooth 60-fps feedback (e.g. a video game, a drawing tool) is a bad fit.
- **Pixel-perfect design.** You can customize with CSS (we did - the pill search bar, filter icon, circular gauges are all custom CSS), but it takes work and some hacks. If your design is the product, use Vercel + React.
- **Multi-page apps with real routing.** Streamlit has multi-page support but it's basic.

- **Heavy user traffic.** Streamlit Cloud's free tier gives your app 1 GB of RAM and modest CPU. Fine for hundreds of daily users, not for millions.

For a portfolio project, none of these matter. Streamlit is exactly right.

Real example searches

These are actual results from the finished app. Score is shown as a percentage relative to the best hit for the query.

1. Query: **vector database embeddings** - Keyword only

Score	Type	Result	Matched words
100	REPO	weaviate/weaviate	vector database, vector, database, embeddings
82	REPO	qdrant/qdrant	vector database, vector, database, embeddings
79	REPO	StarlightSearch/EmbedAnything	vector database, vector, database, embeddings
71	REPO	kreeben/resin	vector database, vector, database
68	REPO	treygrainger/ai-powered-search	vector database, vector, embeddings, database

Notice the top-5 are *all repos*. That's because the phrase "vector database" is dominated by tool projects, not academic papers about them.

2. Query: **conversational search chatbot** - Keyword only

Score	Type	Result	Matched words
100	PAPER	Towards Fair Conversational Recommender Systems	conversational
93	PAPER	Improving Ad-hoc Search Effectiveness for Conversational IR via Model Merging	conversational, search
84	PAPER	Adaptive Personalized Conversational Information Retrieval	conversational, search
63	PAPER	Explainable Information Retrieval in the Audit Domain	conversational, search
60	PAPER	VideolandGPT: A User Study on a Conversational Recommender System	conversational

Now it's *all papers*, because "conversational IR" is a hot academic research area - the papers use that phrase far more densely than any repo README does.

3. Query: **chatbot** - Keyword vs Semantic side by side

Keyword column:

Score	Type	Result	Matched
100	PAPER	<i>Adapting LLMs for Efficient, Personalized IR: Methods and Implications</i>	chatbot
74	REPO	bigscience-workshop/petal	chatbot

Score	Type	Result	Matched
70	REPO	aurelio-labs/semantic-router chatbot	

Semantic column (same query!):

Score	Type	Result	Matched
100	REPO	deeppavlov/DeepPavlov	(matched by meaning)
95	REPO	onyx-dot-app/onyx	(matched by meaning)
92	REPO	RasaHQ/rasa	(matched by meaning)

The interesting thing: the semantic column found the three most famous open-source conversational-agent frameworks in the world - DeepPavlov, Onyx, RasaHQ - none of which contain the word "chatbot" in the text we indexed. The keyword search couldn't find them. This is exactly the moment the "meaning" search earns its keep.

Glossary of every technical term

API (Application Programming Interface). A doorway that a website leaves open for programs to talk to it directly. Instead of a human clicking around in a browser, a script sends a URL and gets back data. arXiv, GitHub, and OpenAI all have APIs.

arXiv. An open online library of research papers, mostly in science, math, and computer science. Free to read, has an API.

Cosine similarity. A number from 0 to 1 that measures how similar two vectors are, by measuring the angle between them. Two identical vectors give 1; two completely unrelated ones give 0. The workhorse of search, recommendation, and clustering.

Dataset. A collection of records that a program will read. Ours is `combined.json` - 832 records, each with a title, source, URL, and text blob.

Embedding. A list of numbers that represents the meaning of a piece of text (or an image, or a song). Two similar pieces of text get similar embeddings. Produced by an AI model that was trained on huge amounts of text. Our embeddings are 384 numbers per document.

GitHub. A website that hosts open-source code projects. Each project is a "repository" (or "repo") with code, a README, and metadata. Has an API.

Hugging Face. A website that hosts free AI models the way GitHub hosts free code. Our sentence-transformers model was downloaded from Hugging Face on first run.

Hybrid search. Running both keyword search and semantic search at the same time, then combining or comparing the results. Modern production search systems (Google, Bing) are all hybrid.

Index. A precomputed data structure that lets you find things quickly, like the index at the back of a textbook. Ours is the TF-IDF matrix.

JSON. A plain-text format for structured data. Every programming language can read it. Files ending in `.json`.

Keyword search. Matching documents to a query based on shared words. The classic approach used by every search engine since the 1970s. Fast, transparent, doesn't understand meaning.

MiniLM (`all-MiniLM-L6-v2`). The small AI model we use for semantic search. Trained by Microsoft, distributed for free by Hugging Face. 90 MB, produces 384-number embeddings, runs on any laptop without a GPU.

NLP (Natural Language Processing). The branch of AI that teaches computers to understand human language - used in search, translation, chatbots, and more.

PyTorch. A library for running AI models. Sentence-transformers uses it under the hood. About 500 MB - the biggest thing we install.

Rate limit. A cap that APIs put on how many requests you can make per hour without logging in. Prevents abuse. Our GitHub fetcher works around it by pulling READMEs from `raw.githubusercontent.com` instead of the API endpoint.

README. A text file (usually `README.md`) at the top of every GitHub repo, introducing the project. Our search treats it as the searchable content for that repo.

Recommender system. A system that suggests items you might like based on what you and others have chosen before. Powers Netflix's "you might also like," Amazon's "customers also bought," Spotify's Discover Weekly.

scikit-learn (`sklearn`). A popular Python library for classical machine learning. We use its `TfidfVectorizer` to build the keyword index and its `cosine_similarity` function to rank results.

Semantic search. Matching documents by *meaning* instead of by exact words. Powered by embeddings. Finds "conversational agent" when you type "chatbot."

Sentence-transformers. A Python library that wraps small, fast AI models for producing embeddings. We use `all-MiniLM-L6-v2`.

Session state. In a Streamlit app, memory that survives across script reruns (which happen after every user interaction). We use it to remember the current query and filter settings.

Streamlit. A Python library that turns a script into a web app without needing HTML, CSS, or JavaScript. See the ["Why Streamlit"](#) section.

TF-IDF (Term Frequency - Inverse Document Frequency). The scoring formula behind our keyword search. Weights each word by how often it appears in a document (TF) and how rare it is across the whole collection (IDF).

Token / Tokenize. Splitting a piece of text into individual words (or word-pieces) so a program can work with them. "Vector database embeddings" tokenizes to `["vector", "database", "embeddings"]`.

Vector. A list of numbers. Every document and every query is a vector inside our search engine.

Vercel. A hosting service, best known for hosting React/Next.js websites. See the [Streamlit vs Vercel](#) section.

Vocabulary. The list of all unique words (and 2-word phrases) our search engine learned by reading the dataset once. Ours has 25,207 entries.

How I'd explain this in an interview

"I built a small search engine called WHY search that indexes 420 research papers from arXiv and 412 open-source projects from GitHub, and searches them together - so a query like 'conversational search' shows you the academic papers *and* the working codebases in the same place. The interesting part is that for every result, it tells you *why* it matched - either the specific words that overlapped or, when I use the semantic mode, the fact that the meaning matched even though the words didn't. I wrote it in Python with scikit-learn for classic TF-IDF and sentence-transformers for the semantic side, and wrapped it in a Streamlit web app with a side-by-side view that lets you visually compare keyword matching against meaning matching for the same query. The project came out of reading Prof. Jie Zou's work on explainable and conversational information retrieval - I wanted a hands-on project where every result shows its own reasoning."

That's four sentences. Practice saying it out loud.

Built with Python, scikit-learn, sentence-transformers, and Streamlit. Inspired by Prof. Jie Zou's research on conversational information retrieval at UESTC.